



Affordable Web Accessibility: A Case for Cheaper ARIA

Yury Puzis, Yevgen Borodin, Andrii Soviak, Valentyn Melnyk, I.V. Ramakrishnan

Department of Computer Science, Stony Brook University, Stony Brook, NY, USA

ABSTRACT

When a commercial entity designs a product, one of the major considerations is the cost of production. No matter how amazing a product idea is, how many features the product has, or how noble the cause is, most products will fail if they are too expensive for the target market or too expensive to compete in the market. Any website is, of course, a product; and web accessibility is often perceived as an added feature that costs extra money to develop. In this paper, we make an argument that the greatest weakness of WAI-ARIA, an accessibility specification for web developers, is the cost of its implementation. We argue that this cost, rather than specific technical constraints, may be the main culprit of the poor accessibility of many websites, except for the most popular ones, and of the slow progress in WAI-ARIA support in screen readers. We contend that the accessibility of the Web could be greatly improved if an *affordable* version of WAI-ARIA were to emerge. This can be achieved by: (a) extending the specification to increase its utility for a wider range of potential users, which will allow for economies of scale, and (b) refining the specification to make it both more cost effective for web development and more open to innovation in user-agent development. In this paper, we discuss accessible web development in the context of cost and identify potential areas of improvement.

Categories and Subject Descriptors

H.5.2 [Information Interfaces and Presentation]: User Interfaces; H.5.4 [Information Interfaces and Presentation]: Hypertext / Hypermedia - navigation

General Terms

Design, Economics, Human Factors, Standardization.

Keywords

Web; accessibility; blind; low vision; browser; screen reader; non-visual; interface; web applications; ARIA; economics.

1. INTRODUCTION

Over the years, web pages have evolved from simple static HTML text files into sophisticated dynamic web applications. In the process, the Web has come to play a major role in our lives. The Web has also made it possible for people with vision impairments to participate more actively in the society.

For web browsing, visually-impaired people employ screen readers (e.g., JAWS, Windows-Eyes, VoiceOver, Dolphin, Sa To Go, NVDA, etc.), which convert the Web to speech, generally

ignoring layout and graphics, and reading aloud all the textual content in web pages. Screen readers enable their users to listen to and navigate web content sequentially in the order it is laid out in the HTML source code, which often does not correspond to visual layout. Screen readers provide many shortcuts to navigate among elements of a particular type, e.g., links, buttons, edit fields, etc. Although not very efficient for web browsing [1], screen readers enable visually-impaired people to browse the Web and perform online activities. Screen-reader behavior can be influenced via webpage metadata (e.g., navigation order, alt text, semantics), and one of the sources of metadata is the Accessible Rich Internet Applications (ARIA) [12] specification developed by W3C.

While the ARIA specification is extensive, the ARIA language is not as expressive as JavaScript, which means that advanced custom webpage behaviors (e.g., Chat widgets) are impossible to make accessible using the ARIA language alone. Fortunately, the ARIA specification enables web developers to control some aspects of screen-reader behavior via webpage JavaScript, by manipulating the screen reader's virtual cursor (the reading position), handling keyboard shortcuts, and providing audio-feedback. In combination with JavaScript, the ARIA language can be used to provide screen readers with webpage semantics and content. At first glance, there seems to be no reason why the ARIA specification cannot be the ultimate solution to all technical web accessibility needs. Yet, 8 years after the first public draft of ARIA was introduced (finalized in 2014), screen readers still implement only a part of the specification, and webpages rarely use ARIA. Many papers have been written on how to evaluate accessibility of websites, provide better development tools, and encourage web developers to make their websites accessible; directives, legislations, standards, tutorials, and guidelines have been published, but many web applications still remain to be only partially accessible.

Then why is the Web not accessible? – One of the important reasons is a high cost of accessibility. Rather than providing a ground-up, comprehensive, end-to-end solution, ARIA was designed to “fill in the gaps” [7, 9] in what was already possible with HTML and JavaScript, paving the way for a nightmarish development and maintenance cycle. And, as the Web grows, the proverbial gap that ARIA was meant to fill only becomes larger.

In this paper, we assert that this situation can be substantially improved as soon as web and user-agent developers realize that it is profitable or, at least, inexpensive (since it is a noble cause). For the moment, ARIA is most useful to screen-reader users, which is why, in this paper, we will assume that user-agents are screen readers. We assert that the cost of making websites accessible can be dramatically reduced by a combination of the following measures. First, refine ARIA to replace the “filling-the-gap” design with an end-to-end cost-effective solution for both web developers and user-agent developers. Second, extend ARIA to enable creation of new categories of user-agents which are useful not just to people with sensory and motor disabilities but also to those who are situationally impaired (e.g., temporarily unable to look at the screen [5]). This change will expand the target audience for web and user-agent developers and reduce the cost by enabling them to create economies of scale. In the remainder of this paper, we will detail the reasons for the high

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
W44'15, May 18–20, 2015, Florence, Italy.
Copyright 2015 ACM 978-1-4503-3342-9/15/05\$15.00.
<http://dx.doi.org/10.1145/2745555.2746657>

cost of making webpages accessible and identify potential areas for improvement.

2. WHAT USERS NEED

2.1 Minimum Viable Screen Reader

We outline the basic needs of screen-reader users below:

- Listen to content and important (relevant) dynamic changes of content in webpages.
- Get feedback about a spatial and semantic relationship among webpage components; the ability to move the virtual cursor among these components, e.g., when in a table, be notified about table dimensions and be able to move the virtual cursor between adjacent cells in all directions.
- Interact with the webpage, e.g., use widgets, fill and submit forms, follow links, etc.
- Access content if and only if it is supposed to be visible; easily find all important (relevant) content. Invisible and “background” content should not be accessible.

The user’s need for information, feedback, and control is both:

- Contextual, i.e. in the context of the current position of the screen reader’s virtual cursor, e.g., if the user is in the “chat” widget, all new chat messages are relevant; and
- Time-sensitive, regardless of the current context, e.g., the time remaining until the end of an online auction.

Screen-reader developers are making an effort to solve those issues in the screen reader without relying on web developers (e.g., guess correct form-field labels in the absence of proper accessibility markup); however, such approaches cannot be 100% accurate because only the web developers know the semantics of the webpage and have the full control over its behavior. Unfortunately, web developers continue to ignore accessibility.

2.2 Example: Accessible Dialog Window

As an illustration of practical issues involved in making a webpage accessible, the following is the list of what the web developer needs to do to create a *very basic* accessible “Log In” dialog window containing a title, “e-mail” and “password” textboxes, as well as “cancel” and “login” buttons:

1. Notify the user that a dialog has opened; read the dialog title.
2. The virtual cursor has to be placed at the most appropriate “starting” point in the dialog, i.e. the “e-mail” form field.
3. Make the user aware that the focus is indeed in the “e-mail” field, and the field can be edited.
4. Add a label to each form field and button, e.g., “e-mail”, “Password”, “Log In”, “Cancel” (see Section 3.3).
5. Make the Tab key iterate through all form fields and buttons in a sensible order, and make the Tab order wrap (go in circle) inside the dialog (see Section 3.6).
6. The user should be prevented from navigating the virtual cursor out of the window: if this happens the user can become “lost” on the webpage and unable to close the dialog. If sighted users cannot interact with the website “under” the window, the same should apply to screen-reader users.
7. Enable the user to control and review (with speech feedback) the content of the form fields. If the textboxes are standard HTML, this will be handled by the screen reader; otherwise, web developers have to do this (see Section 3.2).
8. A shortcut (e.g., the “Esc” button) needs to close the dialog.

9. Form-entry errors (e.g., an unrecognized email address notification) are time-sensitive pieces of information and should be discoverable by the screen-reader user as soon as they are displayed (see Section 3.5).
10. Once the dialog is closed, the virtual cursor should be placed in a sensible position, ideally back to where it was when the dialog opened. Otherwise the user may become “lost.”

Although dialog windows and simple controls are supported in ARIA, implementing the above list will require the collaboration of a technical writer, a web developer, a web designer, and, likely, a web accessibility specialist. The required customizations will encompass HTML, ARIA (hosted by HTML), JavaScript, the server-side code, possibly CSS, etc. The team will have to consider differences between browsers and the ways different screen readers support ARIA. The implementation will need to pass quality assurance and be *maintained throughout the website lifecycle*. When using a third-party development framework the team will face additional hurdles if any modifications of the framework are needed. If making a login window is this complicated and, thus, costly, then the cost of making a sophisticated custom widget (Section 3.2) accessible may be substantially greater.

3. WHAT DEVELOPERS HAVE

As already mentioned, the ARIA specification has been designed to be used by developers of websites and interpreted by screen readers. We next give a detailed, but in no way complete, review of the complexities involved in following the specification.

3.1 ARIA-Defined Widgets

This category includes all widgets (discrete user interface objects with which the user can interact) that have a predefined the ARIA widget “role” (e.g., “tab”, “treeitem”, “checkbox”, etc.). The ARIA-defined widget set subsumes and expands on the standard HTML widgets (e.g., “textbox”, “checkbox”, etc.).

The following ARIA quote: “*assistive technologies ... SHOULD switch to an application browsing mode, and pass keyboard events through to the web application.*” [10] means the web developers will have to make widgets accessible by:

1. Using the ARIA language (hosted in HTML) to flag DOM nodes as widgets and specify widget states and properties.
2. Using JavaScript to make selected shortcuts modify the widget state and update corresponding ARIA attributes in the DOM tree (e.g., set “aria-checked=true” for a “checkbox”).
3. Using JavaScript to make selected shortcuts control the browser focus, which will, in turn, control the virtual cursor – for widgets not defined in HTML; for native HTML widgets (e.g., list box <select>) shortcuts are handled by the browser.

As a result, a web developer is required to implement accessibility-related logic in many languages and in many parts of the website: HTML, ARIA, JavaScript, and optionally CSS, server-side, etc.

3.2 Custom Widgets

This category includes all widgets that do not have an explicit ARIA role, and are instead flagged with the role “application”.

When describing custom widgets, the ARIA specification contains nearly identical wording to that given in Section 3.1. Specifically, web developers have to make those widgets accessible in the same way as ARIA-defined widgets without their HTML counterpart. The main distinction is that the web developers will not be able to

use ARIA markup to provide screen-reader developers with the semantics of the widget. As a result, screen-reader developers will not be able to customize the screen-reader behavior accordingly. As a result, the web developers will have to use additional JavaScript and more general-purpose ARIA markup to make the widget accessible (see Sections 3.3, 3.4, and 3.5).

The ARIA Authoring Practices [8] provide “understanding of how to use [...] ARIA to create accessible rich Internet applications” and can be useful in designing custom widgets by creatively combining HTML, ARIA, and JavaScript. However, the staggering length of the document alone (more than 100 pages, not counting the ARIA specification itself) is evidence to the complexity of the endeavor.

3.3 Speech Feedback

ARIA cannot be used to define explicitly what or when the user-agent should speak. Besides modifying the widget state (e.g., setting “aria-checked” to “true”), web developers can only influence a screen-reader speech output by assigning labels to all accessible webpage elements. Below is a non-exhaustive list of approaches to specifying a label:

1. ARIA allows to define a label in another node (attributes “aria-labeledby”, “aria-describedby”), or specifying the label explicitly in the labeled node (attribute “aria-label”).
2. HTML allows to define a label in another node (tag “label” with attribute “for”) or in a parent “label” node.
3. There are also many non-standard approaches to defining a label with attributes “placeholder”, “tooltiptext”, “description”, “value”, “alt,” using the label of the parent node, or the text in descendant nodes, labels of geometrically adjacent nodes, etc.

A user-agent algorithm for determining labels may account for the type of the labeled node will require recursion, a check for cycles, and an account for dynamic DOM mutations that may lead to computing some or all labels from scratch after every DOM mutation. The task of guessing labels is so complex that even machine-learning methods have been proposed [3].

A web developer also needs to help user-agents give users feedback on state changes of stateful widgets. ARIA-defined widgets can be handled by exposing their state via appropriate ARIA attributes. Custom widgets can only be handled by creatively using JavaScript and ARIA to “trick” the user-agent into providing correct feedback.

As a result, in most cases, the web developer has no way of knowing with certainty what the user will actually hear, while the screen-reader developer has no way of knowing with certainty what the web developer wants the user to hear.

3.4 Perceivable Content

Notifying the user-agent that some content should not be accessible since it is invisible, is in the “background” (e.g., behind an open window), or is not meant for non-visual consumption is another critical step in making a webpage accessible.

There exist a huge number of approaches to hide content from *sighted* webpage visitors by using HTML, JavaScript, and Cascaded Style Sheets (CSS). These include, but are not limited to, modifying the node “style” attribute, “class” attribute, some other attributes, or the class itself; specifically, using “display:none”, setting the color to match the background, setting the font size or object dimensions to 0, setting opacity to full transparency, using “visibility: hidden”, hiding an object behind other objects (using “z-index”), hiding an object beyond the

viewport of the containing object, hiding an object beyond the viewport of the webpage itself, deleting the DOM subtree, etc. In addition, web developers have the option of using the ARIA “aria-hidden” attribute to specify that an object should not be perceivable by user-agent users.

The variety of the methods above and the fact that some are used by *web developers* (HTML/JavaScript programmers) while others – by *web designers* (HTML/CSS programmers) makes it very hard to track down all the cases and correctly assign and maintain the state of the ARIA “aria-hidden” attribute. Therefore, “aria-hidden” is used inconsistently while user-agents look beyond this attribute to determine what should be hidden from the user.

As a result, in most cases, the web developer has no way of knowing with certainty what will be perceived by the user, while the screen-reader developer has no way of knowing with certainty what the web developer wants to be perceived.

3.5 Time-Sensitive Alerts

ARIA defines *live regions* (“perceivable regions of a web page that are typically updated as a result of an external event when user focus may be elsewhere” [11]) that can be used by web developers to provide users with relevant time-sensitive feedback.

Live Regions include special cases (“alert”, “status”, “timer”, “marquee”) and a general case (“aria-live”) with politeness levels (“off”, “polite”, “assertive”) determining the urgency of the changes in the region. The user can further customize the behavior of live regions (“aria-busy”, “aria-atomic”, “aria-relevant”).

Issues with live regions, such as difficulties with determining the causality of region updates, combining technically discrete but semantically atomic updates, handling interim updates, and employing higher-level abstractions, are well summarized in [6]. Most importantly, the concept of live regions is not a perfect fit for all use cases: the state-of-the-art highly dynamic web applications are, essentially, “live” at the scale of the entire webpage. As a result, relevant and time-sensitive dynamic webpage updates are often left unreported to the user-agents.

3.6 Navigation

Important sections of the webpage can be specified both using HTML (e.g., “header”, “aside”, “footer”, etc.) and ARIA (“region”, and more specialized *landmarks*). Page structure can also be specified using approaches unintended by HTML: “div”, “table”, etc., and some widgets have internal structure, e.g., “table” in HTML, “grid” in ARIA, etc. Semantic relationships between page components, in addition to HTML structure, can be specified by a number of ARIA attributes. Overall, there exists a significant amount of specification overlap and non-standard practices in creating webpages, which causes ambiguities in the interpretation of web-developer intent by user-agent developers.

Web developers also can customize the behavior of the screen reader’s virtual cursor (usually controlled with keyboard shortcuts). This can be done both declaratively (attribute “aria-owns”), imperatively (by binding shortcuts to widget functionality and using JavaScript to manipulate the browser focus which is expected to be followed by the virtual cursor), and by using the “tabindex” HTML attribute. In other cases, the behavior of the virtual cursor is determined by the user agent. This creates two problems: (a) manipulating the virtual cursor with browser focus is essentially a hack, since the two have different semantics, and (b) control over the virtual cursor is divided between 3 development teams: user-agent, webpage, and browser (via browser focus, e.g., in standard HTML controls) teams.

3.7 Development Tools

Third-party widget toolkits and development frameworks sometimes provide *limited* ARIA support useful for implementing accessible widgets that are simple and do not need customization (i.e. additional maintenance). This approach reduces the development cost, but the additional cost of developing an accessible framework and the need to compete against frameworks that do not invest in accessibility results in a short list of mostly non-commercial limited frameworks that, so far, did not succeeded in making a dent in overall Web accessibility.

An alternative, available to interested third-parties (e.g., web accessibility enthusiasts), is to fill the webpage accessibility “gap” by injecting the necessary ARIA markup and JavaScript using a browser extension, a bookmarklet, or a proxy server. Injection can be of a) a automatically generated ARIA or b) handcrafted ARIA and JavaScript. SASWAT [4] is an example of the former, designed as a proxy web server. Its downside is the limitations of the ARIA specification and the fact that the automatic modification of a webpage by a third-party tool may lead to unexpected results. AsxJax [2] is an example of the latter. When used by third-party developers, AsxJax stability suffers because webpages are constantly changing and the third-party code injected into the webpage can quickly become outdated.

4. WHAT DEVELOPERS NEED

4.1 Web Development

To summarize, the high cost of web development stems from:

1. Accessibility logic crosses programming language boundaries: by design, HTML hosts the ARIA code and both languages are used to express important semantics; JavaScript and/or the server-side code have to be used to provide non-basic accessibility functionality, etc.
2. Accessibility logic crosses programming paradigm boundaries: some of the logic is declarative and some is imperative.
3. Accessibility logic crosses role boundaries, involving technical writers, web developers, web designers, and, likely, web accessibility specialists.
4. Accessibility logic crosses organization boundaries: the webpage, the user-agent, and the browser, all with different goals and priorities.

As a result, implementation and maintenance require a significantly greater investment in development time in comparison to the implementation done in a single language, by a single organization, etc. Extra development time translates into greater costs. Further, lack of certainty in how the user-agent will behave, even when ARIA is followed to the letter, leads to poor quality control over the webpage accessibility code, which leads to poor quality of screen-reader behavior.

What organizations that are building websites and widget toolkits need is a motivation to do the right thing and a lower complexity/cost to produce quality implementations. This requires the ability to communicate their intent to user-agents both easily and unambiguously. Identifying the exact solution will require an effort far beyond the scope of this paper, but we would like to start by suggesting the following approach. First, move the ARIA Role Model from ARIA to HTML, which will lead to a single way of defining the semantics of webpage structure and widgets – in HTML. This will eliminate the duopoly of ARIA and HTML. Second, extend the ARIA language to enable web developers to *explicitly* define the user-agent behavior (e.g., speech feedback patterns, the virtual cursor movement, etc.) *conditional on page*

element roles and states. Given a sufficiently expressive language, this will remove the need for JavaScript in accessibility logic and eliminate the duopoly of ARIA and JavaScript. Of course, it will still be possible to use non-standard tricks, but it will be unnecessary rather than unavoidable.

4.2 User-Agent and Browser Development

The main cost of user-agent development stems from constraints on the *ability to innovate*. There are too many parties affecting the user-agent behavior: web developers (HTML, ARIA, JavaScript), web designers (HTML, CSS), the browser (Accessibility API, Browser focus, native implementation of HTML widgets), and the user-agent (virtual cursor, shortcuts, widget logic). Coupled with the problems in interpreting the intentions of web developers (Section 3), this places significant constraints on the directions in which the innovation of user-agents can be attempted.

For example, the Browser Accessibility API is a cost-saving feature allowing user-agents to avoid interpreting HTML and ARIA directly; however, browser developers are not financially motivated to invest in improving Accessibility API, limiting, in turn, the utility of the API (e.g., high latency, limited access to the DOM, etc.), and forcing user-agents to go back to basics: reading the DOM tree to create an innovation, e.g., Capti screen reader [5]. In a world where Role Model is moved from ARIA to HTML, Browser Accessibility API will be simpler (cheaper), or even unnecessary, benefitting both browser and user-agent developers.

Of course, supporting an extended ARIA specification also means additional costs for user-agent developers, creating a chicken-and-egg problem. However, extending rather than replacing the ARIA specification will allow for an incremental improvement by both web and user-agent developers, and proactive measures (e.g., by governments, accessibility enthusiasts) should seal the deal.

5. ACKNOWLEDGEMENTS

This material is based upon work supported by the National Science Foundation under Grant No. IIS-1218570.

6. REFERENCES

- [1] Borodin, Y., et al., *More than meets the eye: a survey of screen-reader browsing strategies*, in *W4A 2010*, ACM: Raleigh, NC. p.1-10.
- [2] Chen, C.L. and T.V. Raman, *AsxJAX: a talking translation bot using Google IM: bringing Web-2.0 applications to life*, in *Proceedings of the 2008 International Cross-Disciplinary Conference on Web Accessibility*. 2008ACM: Beijing, China.
- [3] Islam, A.M., Y. Borodin, and I.V. Ramakrishnan, *Mixture Model based Label Association Techniques for Web Accessibility*, in *UIST*. 2010: New York, NY, USA.
- [4] Jay, C., A.J. Brown, and S. Harper, *Internal evaluation of the SASWAT audio browser: method, results and experimental materials*. 2010, The University of Manchester.
- [5] Puzis, Y., et al. *Predictive Web Automation Assistant for People with Vision Impairments*, in *WWW'2013*, ACM: Rio de Janeiro, Brazil.
- [6] Thiessen, P. et al., *Ajax live regions: chat as a case example*, in *Proceedings of W4A 2007*, ACM: Banff, Canada. p. 7-14.
- [7] WAI-ARIA: *Conflicts with Host Language Semantics*. 2014: http://www.w3.org/TR/wai-aria-complete#host_general_conflict.
- [8] WAI-ARIA-Practices. *WAI-ARIA: Practices*. 2013 Available from: <http://www.w3.org/TR/wai-aria-practices/>.
- [9] WAI-ARIA-Roadmap. *WAI-ARIA: Roadmap*. 2008; Available: <http://www.w3.org/TR/wai-aria-roadmap/>.
- [10] WAI-ARIA-Roles. *WAI-ARIA: Roles*. 2014 [cited 2015]; Available from: <http://www.w3.org/TR/wai-aria/roles>.
- [11] WAI-ARIA-Terms. *WAI-ARIA: Important Terms*. 2014; Available from: <http://www.w3.org/TR/wai-aria/terms>.
- [12] WAI-ARIA. *W3C Accessible Rich Internet Applications*. 2013; Available from: <http://www.w3.org/TR/wai-aria>.